

Taller 0 - Primeros pasos con SystemVerilog y HDL Studio

Sistemas Digitales

Segundo Cuatrimestre 2026

1. Enunciado

El taller consistirá en dar los primeros pasos con SystemVerilog y el entorno de la materia: Formulas lógicas, circuitos y su implementación en SystemVerilog.

Para la realización de este taller deberá utilizar el paquete inicial del Taller 0 (devcontainer, esqueletos .sv y testbenches) disponible en la página de la materia.

Se deberá enviar al campus la resolución (el archivo .sv) de al menos 1 ejercicio de este taller. El plazo de entrega finaliza el 19 de agosto a las 23:59.

2. HDL Studio

2.1. El entorno

El trabajo se realiza dentro del entorno provisto por la materia (VS Code + devcontainer + HDL Studio). No es necesario instalar herramientas por fuera del contenedor.

Al abrir la carpeta del taller, aceptar la apertura dentro del devcontainer y verificar que HDL Studio y las extensiones de SystemVerilog estén activas.

El entorno opera en dos modos complementarios. Simulación y síntesis:

- En simulación se ejecuta un testbench, se observan entradas y salidas, y se comprueba el comportamiento del módulo.
- En síntesis se visualiza el circuito inferido a partir del código y se lo relaciona con las expresiones escritas.

2.2. Detalles adicionales

- a) Las entradas (**input**) se declaran en la interfaz del módulo y reciben valores desde el testbench o desde otros módulos.
- b) Las salidas (**output**) se declaran en la interfaz y deben recibir una asignación en el cuerpo del módulo.
- c) Las señales internas se declaran con **logic** dentro del módulo y conectan subexpresiones.
- d) Se recomienda fuertemente usar los nombres del enunciado para puertos y señales, de modo que coincidan con los testbenches.
- e) Los testbenches son parte de la especificación: pueden leerse, pero no deben modificarse para lograr que una implementación incorrecta sea aceptada, salvo donde el enunciado pida completar casos de prueba.

3. Antes de empezar

1. Abrir la carpeta del taller en VS Code.
2. Aceptar la apertura dentro del devcontainer.
3. Verificar que HDL Studio y las extensiones de SystemVerilog estén activas.
4. Abrir el ejemplo provisto y ejecutar su testbench.
5. Identificar en pantalla las entradas, las salidas y el circuito sintetizado.

Si el entorno no funciona, registrar el mensaje completo del error y consultar antes de instalar herramientas por fuera del contenedor.

4. Ejercicios

Completar el esqueleto provisto de los módulos que se enumeran a continuación. Deberán diseñar cada uno de manera modular, incorporando una funcionalidad por módulo.

4.1. Ejercicio 1: Abrir y explorar un ejemplo.

En la carpeta `ejercicios/t0_ej1` se provee el módulo `compuerta_misterios`: una entrada `a`, una salida `y`, y una asignación misteriosa.

```
module compuerta_misterios (  
    input  logic a,  
    output logic y  
);  
    assign y = ???;  
endmodule
```

- Abrir `compuerta_misterios.sv` y explicar qué hace cada línea: puertos, `assign` y el operador usado.
- Abrir el circuito en HDL Studio (*Create circuit in HDL studio*) y sintetizarlo (*Synthesize*).
- Identificar en la vista sintetizada la entrada, la salida y la compuerta misteriosa inferida.
- Clicar la entrada en la vista del circuito y observar cómo cambia la salida.
- Ejecutar el testbench (`make sim`) y comprobar que los dos casos posibles pasan.

4.2. Ejercicio 2: Señales internas.

En la carpeta `ejercicios/t0_ej2`, implementar la función

$$y = (a \wedge b) \vee (\neg a \wedge c)$$

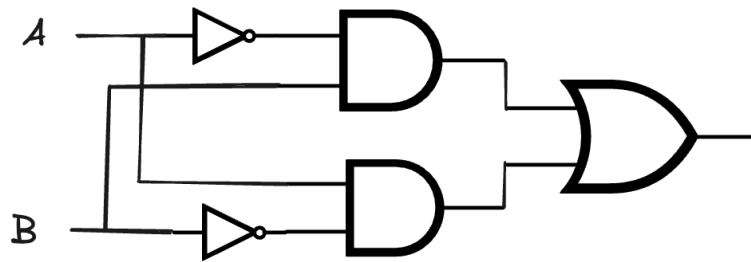
usando dos señales internas llamadas `p` y `q`. Cada señal debe representar una subexpresión. Comparar la estructura sintetizada con la expresión original.

```
module circuito_ej2 (  
    input  logic a,  
    input  logic b,  
    input  logic c,  
    output logic y  
);  
    logic p, q;  
    // completar los assign  
endmodule
```

- Calcular a mano la salida para `a b c = 101`.
- Ejecutar el testbench (`make sim`) y comprobar que los casos pasan.
- Abrir la vista sintetizada y corroborar el cálculo realizado para las entradas `101`.

4.3. Ejercicio 3: Interpretar un circuito.

En la carpeta `ejercicios/t0_ej3` se provee el cascarón `circuito_ej3`. El siguiente diagrama está construido con compuertas AND, OR y NOT:



- Escribir la fórmula lógica que representa el diagrama.
- Completar `circuito_ej3.sv` implementando esa fórmula.
- Completar los 4 casos esperados en el testbench `ejercicios/t0_ej3/circuito_ej3_tb.sv`, en el bloque marcado `Completar acá los 4 casos esperados de y` (entradas `00`, `01`, `10` y `11`).
- Ejecutar el testbench (`make sim`) y comprobar que los 4 casos pasan.
- Analizar el circuito y su fórmula lógica. ¿Se puede reescribir de otra manera? Si es así, escribir la nueva fórmula lógica y el nuevo código en SystemVerilog, y verificar que los 4 casos de prueba pasan.